```python
# coding:utf-8
import numpy as np
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

def pca(data, n_components):
    '''
    pca is O(D^3)
    :param data: (n_samples, n_features(D))
    :param n_dim: target dimensions
    :return: (n_samples, n_components)
    '''
    # 减去均值，也可以进行标准化处理（除以方差）
    data = data - np.mean(data, axis = 0, keepdims = True)
    m,_ = data.shape
  # 计算协方差矩阵
    cov_matrix = 1/m * np.matmul(data.T, data)
    # 计算协方差矩阵特征值及对于的特征向量
    eig_values, eig_vector = np.linalg.eig(cov_matrix)
    # print(eig_values)
    # 对特征值进行排序，并取前n_components组
    indexs_ = np.argsort(-eig_values)[:n_components]
    picked_eig_vector = eig_vector[:, indexs_]
    data_ndim = np.matmul(data, picked_eig_vector)
    return data_ndim


# data 降维的矩阵(n_samples, n_features)
# n_dim 目标维度
# fit n_features >> n_samples, reduce cal
if __name__ == "__main__":
    data = load_iris()
    X = data.data
    Y = data.target
    pca_line = PCA().fit(X)
    plt.plot([1,2,3,4],np.cumsum(pca_line.explained_variance_ratio_))
    plt.xticks([1,2,3,4]) #这是为了限制坐标轴显示为整数
    plt.xlabel("number of components after dimension reduction")
    plt.ylabel("cumulative explained variance ratio")
    plt.show()

    data_2d1 = pca(X, 2)
    plt.figure(figsize=(15,4))

    plt.subplot(121)
    plt.title("my_PCA")
    plt.scatter(data_2d1[:, 0], data_2d1[:, 1], c = Y)

    sklearn_pca = PCA(n_components=2)
    data_2d2 = sklearn_pca.fit_transform(X)
    plt.subplot(122)
    plt.title("sklearn_PCA")
    plt.scatter(data_2d2[:, 0], data_2d2[:, 1], c = Y)
    plt.show()
```